CISS245: Advanced Programming
Quiz q7101

Name: eagraham1@cougars.ccis.edu Score:

Q1. The following program uses the `vec2d` library.

```
#include <iostream>
#include "vec2d.h"

int main()
{
    vec2d v(1.2, 3.4);

    int i = 2;
    try
    {
        std::cout << v[0] << std::endl; // prints 1.2
        std::cout << v[1] << std::endl; // prints 3.4
        std::cout << v[i] << std::endl; // exception thrown if i < 0 or i > 1
    }
    catch (IndexError & e)
    {
        std::cout << "IndexError caught" << std::endl; // prints IndexError caught
    }
    return 0;
}
```

Modify the `vec2d.h` file so that the exception catching code runs when the value of `i` is not `0` or `1`.
Answer:

```
// file: vec2d.h
#ifndef VEC2D_H
#define VEC2D_H

class IndexError
{};

class vec2d
{
public:
    vec2d(double x, double y)
        : x_(x), y_(y)
    {}
    double operator[](int i) const
```

```
    {
        if (i > 1 || i < 0) throw IndexError();
        return (i == 0 ? x_ : y_);
    }
    // other methods not shown
private:
    double x_, y_;
};
```

Q2. Modify the given `Rational.h` so that exceptions are thrown whenever there is a denominator of zero.
Answer:

```
// file: Rational.h
#ifndef RATIONAL_H
#define RATIONAL_H

class ZeroDenominatorError
{};

class Rational
{
public:
    Rational(int n, int d)
        : n_(x), d_(d)
    {}
    const Rational & operator/=(const Rational & f) const
    {
        if (f.d == 0 || d_ == 0) throw ZeroDenominatorError();
        n_ /= f.d_;
        d_ /= f.n_;
        return (*this);
    }
    Rational operator/(const Rational & f) const
    {
        if (f.d == 0 || d_ == 0) throw ZeroDenominatorError();
        return (Rational(*this) /= f);
    }
private:
    int n_, d_;
};
```

(Hint: Modify at two places.)

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.